一、对象

1.1 介绍

介绍： JavaScript里的一种数据类型，可以理解为是一种无序的数据集合， 注意数组是有序的数据集合

- 用来描述某个事物，例如描述一个人
- 人有姓名、年龄、性别等信息、还有吃饭睡觉打代码等功能
- 如果用多个变量保存则比较散，用对象比较统一
- 比如描述 班主任 信息：
  - 静态特征 (姓名, 年龄, 身高, 性别, 爱好) => 可以使用数字, 字符串, 数组, 布尔类型等表示
  - 动态行为 (点名, 唱, 跳, rap) => 使用函数表示

1.2 使用

- **对象有属性和方法组成**
  - 属性：信息或叫特征（名词）。 比如 手机尺寸、颜色、重量等...
  - 方法：功能或叫行为（动词）。 比如 手机打电话、发短信、玩游戏...

```
let 对象名 = {
    属性名: 属性值,
    方法名: 函数
}
```

- **属性**
  - 数据描述性的信息称为属性，如人的姓名、身高、年龄、性别等，一般是名词性的

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男'
}
```

  - 属性都是成 对出现的，包括属性名和值，它们之间使用英文 : 分隔

- 多个属性之间使用英文，分隔
- 属性就是依附在对象上的变量（外面是变量，对象内是属性）
- 属性名可以使用 "" 或 ''，一般情况下省略，除非名称遇到特殊符号如空格、中横线等

1.3 属性的增删改查

1.3.1 查

语法： `对象.属性`

作用： 获得对象里面的属性值

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男'
}

console.log(obj.name);
console.log(obj.age);
console.log(obj.gender);
```

1.3.2 改

语法： `对象名.属性 = 新值`

作用： 修改对象属性值

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男'
}
obj.name = '李四'
console.log(obj.name);
```

1.3.3 增

语法： `对象名.新属性 = 新值`

作用： 增加新的属性

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男'
}
obj.hieght = 180
```

1.3.4 删

语法： `delete 对象名.属性`

作用： 删除属性

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男'
}
delete obj.gender
console.log(obj);
```

1.3.5 属性-查的另外一种写法

- 对于多词属性或着包含 - 等属性，点操作就不能用了。
- 我们可以采取： 对象[‘属性’] 方式， 单引号和双引号都阔以

```
let obj ={
    'user-name': '张三',
    age: 19,
    gender: '男'
}

console.log(obj['user-name']);
```

- 对象['属性'] 方式， 单引号和双引号都阔以
- 也可以用于其他正常属性，比如：

```
let obj ={
    'user-name': '张三',
    age: 19,
    gender: '男'
}

console.log(obj['age']);
```

  - 语法里面的值如果不添加引号 默认会当成变量解析

总结：

没有必要的时候直接使用点语法，在需要解析变量的时候使用 [ ] 语法

1.4 对象方法

介绍： 数据行为性的信息称为方法，如跑步、唱歌等，一般是动词性的，其本质是函数。

```
let obj ={
    name: '张三',
    sayHi: function() {
        console.log('hi~~');
    }
}

obj.sayHi()
```

1. 方法是由方法名和函数两部分构成，它们之间使用 : 分隔
2. 多个属性之间使用英文 , 分隔
3. 方法是依附在对象中的函数
4. 方法名可以使用 "" 或 ''，一般情况下省略，除非名称遇到特殊符号如空格、中横线等

1.5 遍历对象

for 遍历对象的问题：

- 对象没有像数组一样的length属性，所以无法确定长度
- 对象里面是无序的键值对, 没有规律， 不像数组里面有规律的下标

```
let obj ={
    name: '张三',
    age: 19,
    gender: '男',
    height: 190
}

for (let k in obj) {
    console.log(k);  // 打印属性名
    console.log(obj[k]); // 打印属性值
}
```

- 一般不用这种方式遍历数组、主要是用来遍历对象
- for in语法中的 k 是一个变量, 在循环的过程中依次代表对象的属性名

- 由于 k 是变量, 所以必须使用 [ ] 语法解析
- 一定记住： k 是获得对象的**属性名**， **对象名**[k] 是获得 **属性值**

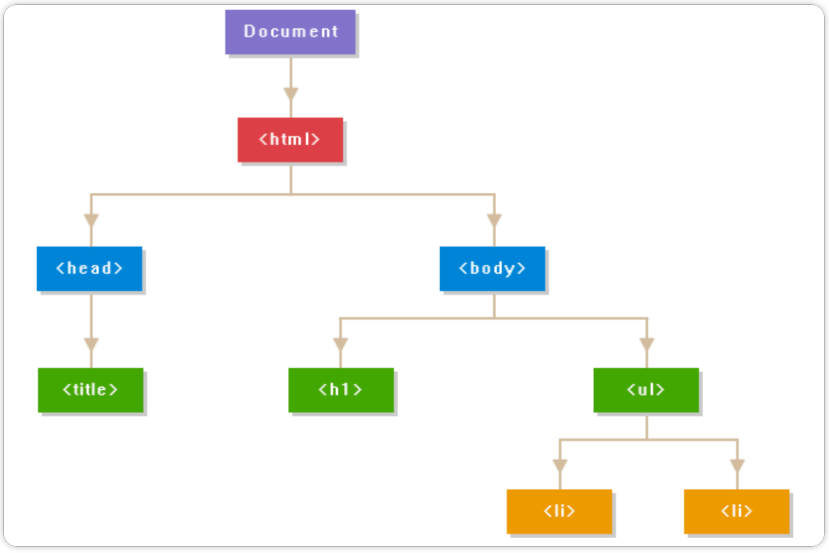
二、DOM

2.1 什么是DOM

介绍： DOM（Document Object Model——文档对象模型），DOM是浏览器提供的一套专门用来 **操作网页内容** 的功能，开发网页内容特效和实现用户交互。

DOM树：

- 将 HTML 文档以树状结构直观的表现出来，我们称之为文档树或 DOM 树
- 描述网页内容关系的名词
- 作用：文档树直观的体现了标签与标签之间的关系



2.2 DOM对象（重要）

2.2.1 对象是什么?

在编程中，**对象** 就像是一个“容器”，它可以装很多东西。这些东西可以是：

- **属性**： 用来描述对象的特征（比如颜色、大小、名字等）。
- **方法**： 用来描述对象能做什么（比如跑、跳、说话等）。

你可以把对象想象成现实生活中的一样东西，比如一只猫、一辆车、或者一个人。

- DOM对象：浏览器根据html标签生成的 JS对象
  - 所有的标签属性都可以在这个对象上面找到
  - 修改这个对象的属性会自动映射到标签身上
- DOM的核心思想
  - 把网页内容当做对象来处理
- document 对象
  - 是 DOM 里提供的一个**对象**
  - 所以它提供的属性和方法都是**用来访问和操作网页内容的**
    - 例：`document.write()`
  - 网页所有内容都在document里面

2.3 小结

1. DOM 树是什么？
   - 将 HTML 文档以树状结构直观的表现出来，我们称之为文档树或 DOM树
   - 作用：文档树直观的体现了标签与标签之间的关系
2. DOM对象怎么创建的？
   - 浏览器根据html标签生成的 JS对象（DOM对象）
   - DOM的核心就是把内容当对象来处理
3. document 是什么？
   - 是 DOM 里提供的一个**对象**
   - 网页所有内容都在document里面

三、获取DOM元素

3.1 根据CSS选择器来获取DOM元素 (重点)

1. **选择匹配一个元素**

语法：

```
document.querySelector('css选择器')
```

参数：

包含一个或多个有效的CSS选择器 `字符串`

返回值:

CSS选择器匹配的第一个元素, 返回一个HTMLElement对象。如果没有匹配到，则返回null。

2. **匹配多个元素**

语法:

```
document.querySelectorAll('css选择器')
```

参数:

包含一个或多个有效的CSS选择器 **字符串**

返回值:

CSS选择器匹配的NodeList 对象集合

例如:

```
document.querySelectorAll('ul li')
```

> **注意:**
>
> ```
> document.querySelectorAll('css选择器')
> ```
>
> 得到的是一个**伪数组**:
>
> - 有长度有索引号的数组
> - 但是没有 pop( )、 push( ) 等数组方法
>
> 想要得到里面的每一个对象，则需要遍历（for）的方式获得
>
> 哪怕只有一个元素，通过querySelectAll( ) 获取过来的也是一个**伪数组**，里面只有一个元素而已

3.2 小结

1. 获取一个DOM元素我们使用谁？能直接操作修改吗?
   - `querySelector()`
   - 可以

2. 获取多个DOM元素我们使用谁？能直接修改吗？如果不能可以怎么做到修改
    - `querySelectorAll()`
    - 不可以， 只能通过遍历的方式一次给里面的元素做修改

3.3 练习

需求： 依次输出3个li的DOM对象

```
<ul class="nav">
    <li>我的首页</li>
    <li>产品介绍</li>
    <li>联系方式</li>
</ul>
```

四、操作元素内容

目标： 能够修改元素的文本更换内容

- DOM对象都是根据标签生成的,所以操作标签,本质上就是操作DOM对象。
- 就是操作对象使用的点语法。
- 如果想要修改标签元素的里面的**内容**，则可以使用如下几种方式：
- 学习路径：

1. 对象.innerText 属性
2. 对象.innerHTML 属性

4.1 innerText属性

作用：

- 将文本内容添加/更新到任意标签位置
- 显示纯文本，不解析标签

```
const info = document.querySelector('.info')
// 获取标签内部的文字
// console.log(info.innerText)
// 添加/修改标签内部文字内容
info.innerText = '嗨喽,俺是刘德华~ '
```

4.2 innerHTML属性

作用：

- 将文本内容添加/更新到任意标签位置
- 会解析标签，多标签建议使用模板字符

```
const info = document.querySelector('.info')
// 获取标签内部的文字
// console.log(info.innerHTML)
// 添加/修改标签内部文字内容
info.innerHTML = `嗨喽,俺是<strong>刘德华</strong>~ `
```

4.3 小结

- 元素.innerText 属性 只识别文本，不能解析标签
- 元素.innerHTML 属性 能识别文本，能够解析标签

五、事件监听（绑定）

介绍：

事件是在编程时系统内发生的**动作**或者发生的事情，比如用户在网页上**单击**一个按钮

5.1 事件监听

什么是事件监听？

就是让程序检测是否有事件产生，一旦有事件触发，就立即调用一个函数做出响应，也称为 绑定事件或者注册事件，比如鼠标经过显示下拉菜单，比如点击可以播放轮播图等等。

语法：

```
元素对象.addEventListener('事件类型', 要执行的函数)
```

事件监听三要素：

- **事件源：** 确定哪个dom元素被事件触发了，要获取dom元素
- **事件类型：** 用什么方式触发，比如鼠标单击 click、鼠标经过 mouseover 等
- **事件调用的函数：** 要做什么事

示例：

```
<button>按钮</button>
<script>
  const btn = document.querySelector('.btn')
  // 修改元素样式
  btn.addEventListener('click', function () {
    alert('点击了~')
  })
</script>
```

> **注意：**
>
> 1. 事件类型要加引号
> 2. 函数是点击之后再去执行，每次点击都会执行一次

5.2 事件类型

鼠标事件	焦点事件	键盘事件	表单事件	窗口/文档事件
click 单击鼠标左键	focus 获的焦点	keydown 按下任意键（持续按住会重复触发）	input 输入框内容实时改变（如输入、粘贴）	load 页面或资源（如图片）加载完成
mouseenter 鼠标经过	blur 失去焦点	keyup 键盘抬起触发	focus 元素获得焦点	resize 浏览器窗口大小改变
mouseleave 鼠标离开			blur 元素失去焦点	scroll 页面或元素滚动时
dblclick 双击鼠标左键				

5.3 事件对象

事件对象是什么？

- 也是个对象，这个对象里有事件触发时的相关信息
- 例如：鼠标点击事件中，事件对象就存了鼠标点在哪个位置等信息
- 如 `event.key`（按键值）、`event.clientX`（鼠标位置）、`event.touches`（触摸点列表）等，用于获取事件详情

使用场景：

- 可以判断用户按下哪个键，比如按下回车键可以发布新闻
- 可以判断鼠标点击了哪个元素，从而做相应的操作

语法：

- 在事件绑定的回调函数的第一个参数就是事件对象
- 一般命名为event、ev、e

```
元素.addEventListener('click', function(e) {

})
```

事件对象

部分常用属性

- type
  - 获取当前的事件类型
- clientX/clientY
  - 获取光标相对于浏览器可见窗口左上角的位置
- offsetX/offsetY
  - 获取光标相对于当前DOM元素左上角的位置
- key
  - 用户按下的键盘键的值
  - 现在不提倡使用keyCode